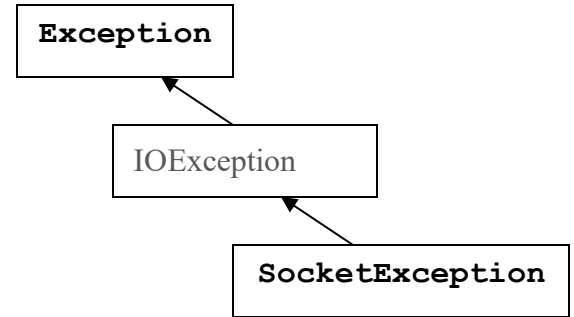


### Question 1

The inheritance hierarchy diagram on the right shows the class relationships between various exception classes. For the **try** block below, if we wish to **catch** each type of exception object separately, in what order, top to bottom, may we place these **catch** blocks.

```
try
{
    Scanner keyboard = new Scanner(System.in);
    int count = keyboard.nextInt();
    if      (count == 1) throw new IOException();
    else if (count == 2) throw new Exception();
    else
        throw new SocketException();
}
```



### Question 2

Circle the letters next to the corresponding statements below that are always *false*.

- a) A non-**static** method from **ClassX** may call another non-**static** method from **ClassX**.
- b) A non-**static** method from **ClassX** may call a **public static** method from **ClassZ**.
- c) A non-**static** method from **ClassX** may call a **static** method from **ClassX**.
- d) A non-**static** method from **ClassX** may directly reference a **static** variable declared inside **ClassX**.
- e) A **static** method from **ClassX** may directly reference an instance variable declared inside **ClassX**.
- f) A **static** method from **ClassX** may call a non-**static** method from **ClassX** using the **this** keyword for referencing (for example: **this.method()** ).
- g) A **static** method from **ClassX** may call a non-**static** method from **ClassX** using the class name for referencing (for example: **ClassX.method()** ).

### Question 3

Give the output of the following syntactically-correct program, assuming the `ExceptionType` classes are subclasses of `Exception`.

```
class ExceptionType1 extends Exception{}
class ExceptionType2 extends Exception{}
class ExceptionType3 extends Exception{}
public class ExceptionTest {
    public static void main(String[] args) throws Exception {
        try {
            A();
            B();
            System.out.println("main-try1");
        } catch (ExceptionType1 e) {
            System.out.println("main-catch1");
        } catch (ExceptionType2 e) {
            System.out.println("main-catch2");
        } catch (ExceptionType3 e) {
            System.out.println("main-catch3");
        } finally {
            System.out.println("mainFinally");
        }
    }
    public static void A() throws Exception {
        try {
            System.out.println("A1-try");
            C();
            System.out.println("A2-try");
        } catch (ExceptionType2 e) {
        } finally {
            System.out.println("A-finally");
        }
    }
    public static void B() throws Exception {
        System.out.println("B-one");
        throw new ExceptionType2();
    }
    public static void C() throws Exception {
        System.out.println("C-one");
        throw new ExceptionType3();
    }
}
```

Output:

Line 1: \_\_\_\_\_

Line 2: \_\_\_\_\_

Line 3: \_\_\_\_\_

Line 4: \_\_\_\_\_

Line 5: \_\_\_\_\_

#### Question 4

a. Analyze the following code and circle the letter of one choice.

```
public class Test {  
    public static void main(String[] args) {  
        B b = new B();  
        b.m(5);  
        System.out.println("i is " + b.i);  
    }  
}
```

```
class A {  
    int i;  
    public void m(int i)  
    {  
        this.i = i;  
    }  
}  
class B extends A {  
    public void m(String  
s) {  
    }  
}
```

- A. The program has a compilation error, because m is overridden with a different signature in B.
- B. The program has a compilation error, because b.m(5) cannot be invoked since the method m(int) is hidden in B.
- C. The program has a runtime error on b.i, because i is not accessible from b.
- D. The method m is not overridden in B. B inherits the method m from A and defines an overloaded method m in B.

b. Which one of the following statements is true?

- A. At least one constructor must always be defined explicitly.
- B. Every class has a default constructor.
- C. Multiple constructors can be defined in a class.
- D. A subclass must implement at least one constructor.

c. What is the output of the following code?

```
public class Test {  
    public static void main(String[] args) {  
        new Human().printHuman ();  
        new Student().printHuman ();  
    }  
}
```

```
class Human {  
    private String getInfo() {  
        return "Human";  
    }  
    public void printHuman() {  
        System.out.println(getInfo());  
    }  
}
```

```
class Student extends Human {  
    private String getInfo() {  
        return "Student";  
    }  
}
```

### Question 5

What is printed by the following code?

```
class C1 extends C3 { }
class C2 extends C3 { }
class C3 extends C4 { }
class C4 extends Object { }
public class Test {
    public static void main(String[] args) {
        Object c1 = new C1();
        Object c2 = new C2();
        Object c3 = new C3();
        Object c4 = new C4();
        System.out.println(c1 instanceof C2); // _____
        System.out.println(c2 instanceof C1); // _____
        System.out.println(c3 instanceof C1); // _____
        System.out.println(c3 instanceof C2); // _____
    }
}
```